

An Evaluation of The Performance of Lightweight Language Models on Zero-Shot NL2SQL tasks

Tom Fan
Carleton University
Ottawa, Canada
tomfan3@cmail.carleton.ca

ACM Reference Format:

Tom Fan. 2025. An Evaluation of The Performance of Lightweight Language Models on Zero-Shot NL2SQL tasks. In . ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 ABSTRACT

Since their introduction as a consumer product in 2022, transformer-based large language models have been used for automating many kinds of digital tasks. This includes natural language to code and SQL tasks, which have exploded in research and development in the last few years. While a large proportion of LLM use is likely done using proprietary, closed source, and large footprint foundation models hosted by SAAS providers like Anthropic or OpenAI, an increasing amount of open-source models are also entering the market. These models have demonstrated promise on several different kinds of tasks, such as code generation using Python [3]. However, there is limited research on how they perform in other common coding-adjacent tasks, including NL2SQL. While there is an emerging wealth of assessment on how larger models like OpenAI's closed source GPT series perform on NL2SQL tasks in various configurations[5][4], the performance of these small models' smaller cousins in non-pretrained configurations is still an under-explored area. In this paper, we aim to provide an overview of how several such opensource models perform when faced with NL2SQL tasks on the Spider-Dev question dataset.

2 INTRODUCTION

In recent years, there has been a significant amount of high performing open source LLMs available from both research institutions and tech companies [6]. While many of the top-ranking models require expensive and complex hardware setups to run as their parameter counts run in the hundreds of billions or even trillions, many are either small footprint by design or distilled versions of larger source models.

In these cases, consumer level hardware should be able to run the models using tools like localLlama, enabling their use offline and without dependency on external SAAS providers (for purposes of this paper, we define consumer level hardware to be current

generation Nvidia RTX chips or other equivalents, able to comfortably run models in the 20B parameter range or smaller at 8 or 16b quantization).

These LLMs have shown promise in their ability to solve an array of tasks, including completing test sets for code generation such as HumanEval [3][2]. In these experiments, the authors were able to push these models to perform disproportionately well to how large their footprints are, indicating that these smaller LLMs are able to consistently solve simple, well defined tasks and generate effective code.

At the same time, there has been significant development in the field of NL2SQL tools. While previous generations of NL2SQL tools depended on hardcoded rules or deep neural networks [4], the emergence of LLMs has enabled a whole new paradigm of NL2SQL tools based on them. In fact, in their MVP state, a LLM itself can serve as a NL2SQL tool on its own.

While there are many evaluations of the potential of LLMs for NL2SQL tasks, many of these depend on the aforementioned large, closed-source, proprietary models [7][4][5]. At the same time, evaluations of the performance of lightweight, open source models are few in number. Therefore, in this paper we conduct an evaluation asking one guiding question - how well do lightweight LLMs such as Llama 8b or Qwen3 perform when presented with NL2SQL tasks in a zeroshot format?

To answer this guiding question, we took a sample of 10 such models in the 0-20b parameter range and evaluated their performance on the SPIDER-dev NL2SQL question set. Using a zero-shot method with a short context length, we built a testbench[8] to evaluate these opensource models using Novita's online AI service and hardware rental service [1]. This allowed us to run each model on the cloud with consistent hardware, while still being able to evaluate token efficiency and getting results equivalent to what we would see running these models on a local system.

Then, we gave each model we evaluated all 1034 of the SPIDER-dev NL2SQL questions, and recorded their responses. Models were not given pre-training or any other context. We found that while all of these models can zero-shot these NL2SQL problems roughly $\frac{1}{2}$ to $\frac{2}{3}$ of the time in terms of execution accuracy, current small opensource LLMs are still not powerful enough to allow this type of use case consistently.

3 TESTING METHODOLOGY AND EXPERIMENTAL DESIGN

We started our experiment with the objective of evaluating the performance of lightweight open source models on NL2SQL tasks. In order to do so, we used SPIDER-dev as our test set (taoyds/spider) [9], which is a commonly used benchmark for evaluation of NL2SQL

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

tools. Then, we evaluated the extracted SQL from the LLMs' responses using the accompanying taoyds/test-suite-sql-eval tool [10], as per the recommendations of SPIDER's authors [9]. To support these requirements, we built a testbench[8] that would serve as a bridge and data recording tool for SPIDER, the evaluator, as well as Novita, our LLM provider.

While the general architecture of our testing system is fairly straightforward, tradeoffs had to be made in experiential design in several areas. These primarily happened in terms of model selection, test set selection, and prompting strategy and arose from a need for testing consistency and efficiency.

Model selection

Our experimental data incorporates testing from 10 different open source LLMs, each with a parameter count $\leq 30b$. The basic criteria for evaluation included the following requirements:

- Models must be open source, or otherwise downloadable and usable for a user locally at no charge and with no external services. This constrains the selected models to effectively what can be found on HuggingFace.
- Models cannot be fine-tuned specifically for NL2SQL tasks, as this would result in imbalanced results and go against the core evaluation of small, general purpose models being used for NL2SQL tasks. Other fine tuning, however, would not be a deal breaking factor for selection.
- Models must be able to run locally on consumer level hardware. While that which constitutes the category of consumer level hardware is subjective to the end user, for the purposes of this paper we have defined it as current generation consumer CPUs (Intel i series, AMD Ryzen, etc) alongside current generation Nvidia RTX graphics processors or equivalent. This constrains the size of the model to 20-30GB total,

as most local LLMs will require $\geq 2GB$ of memory for KV cache or buffer space.

Using this selection criteria, we selected and tested a list of these 12 models:

•

4 RESULTS

5 ANALYSIS

6 CONCLUSION

REFERENCES

- [1] Novita AI. [n. d.]. Novita AI. <https://novita.ai/>. As of Dec 2025.
- [2] A. Corradini, F. Martino, and P. Rossi. 2025. A Comprehensive Survey of Small Language Models in the Era of Large Language Models. *Big Data and Cognitive Computing* 9, 7 (2025), 189. <https://doi.org/10.3390/bdcc9070189>
- [3] T. Hasan, Y. Li, and M. Chen. 2025. Assessing Small Language Models for Code Generation. *arXiv preprint arXiv:2507.03160* preprint (2025), 1 – 17. <https://arxiv.org/abs/2507.03160>
- [4] Boyan Li, Yuyu Luo, Nan Tang, Guoliang Li, and Chengliang Chai. 2024. The Dawn of Natural Language to SQL: Are We Fully Ready? *Vldb* 17, 11 (2024), 3318 – 3331. <https://www.vldb.org/pvldb/vol17/p3318-luo.pdf>
- [5] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2024. A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going? *arXiv preprint arXiv:2408.05109* 18, 9 (2024), 1 – 20. <https://doi.org/10.48550/arXiv.2408.05109>
- [6] llm stats.com. 2025. LLM Leaderboard. <https://llm-stats.com/>. As of Dec 2025.
- [7] Wenqi Pei, Hailing Xu, Hengyuan Zhao, Shizheng Hou, Han Chen, Zining Zhang, Pingyi Luo, and Bingsheng He. 2025. Feather-SQL: A Lightweight NL2SQL Framework with Dual-Model Collaboration Paradigm for Small Language Models. *arXiv preprint, 17811* (2025), 1 – 20. <https://arxiv.org/abs/2503.17811>
- [8] GitHub Repository. [n. d.]. Source Code for Experiment Report. <https://github.com/CATBOYCOWBOY/comp5118Final.git>.
- [9] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium.
- [10] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suite. In *The 2020 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics.